Session 6 · Deck 6.A2

Aiyagari II: From a Policy to a Population, and a Price

Zhigang Feng

Thursday, October 29, 2026

© Zhigang Feng Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 6.A2 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — Aiyagari: 6.A1 the household, from Bellman to Euler · **6.A2** the population and the price.
Block B: 6.B1 Krusell–Smith · 6.B2 transitions and the life cycle.

© Zhigeng Fang
Redistribution prohibited without permission

Roadmap: Six Parts

6.A1 delivered a saving policy at one price. This deck turns it into a population, and the population into a price.

Part	Topic	The question it answers
1	One Markov chain on the grid	How does a policy move a population, and what do we solve for?
2	Mass that lands between nodes	What happens when a' falls between two nodes?
3	Two ways to the fixed point	Iterate the operator, or solve a linear system once: what each costs.
4	Why it works, and how to simulate	Why one answer; how a simulated panel does the same job, at what cost.
5	Clear the market	The price: a sign-checked bisection, or steps along the excess demand.
6	The equilibrium, and what it says	Is it right, and what does it get right about inequality?

© Zhigang Feng
Reproduction prohibited without permission

Idea

Given a policy, the population is the fixed point of a linear map; given the population, the price is the root of one scalar function. Both steps are simple, and each has one trap.

1 · One Markov Chain on the Grid

A household at (a_i, ε_q) saves $a'(a_i, \varepsilon_q)$ and draws ε' from Π. The policy and the chain move every household; the population is where that motion stops.

The operator. Mass $\lambda(a, \varepsilon)$ today becomes, tomorrow,

$$\lambda'(a', \varepsilon') = \sum_{\varepsilon} \sum_{a:\, a'(a,\varepsilon)=a'} \lambda(a, \varepsilon)\, \Pi(\varepsilon, \varepsilon').$$

This is the Markov operator Q of 2.A3. On the grid the state is the *pair* (a_i, ε_q), $200 \times 7 = 1{,}400$ states, so P_g is $1{,}400 \times 1{,}400$ and carries both the policy (which asset node) and Π (which ε'); one step is $\lambda' = \lambda P_g$.

The object. The invariant distribution solves

$$\lambda = \lambda P_g, \qquad \sum \lambda = 1.$$

Idea

The policy is fixed now: this is a *linear* problem, unlike 6.A1's.

Route	What it does	Part
iterate	$\lambda_{n+1} = \lambda_n P_g$ until it stops	3
linear solve	$(P_g^\top - I)\lambda = 0$, $\sum \lambda = 1$	3
simulate	a panel of households	4: how, and the cost

But first a problem every route shares: $a'(a_i, \varepsilon_q)$ is almost never a node. Where does the household's mass go?

© Zhigang Feng · Redistribution prohibited without permission

Notation as fixed in Session 2: λ the invariant distribution, Q the operator on measures.

Off the Grid: Rounding Moves the Mean, the Lottery Does Not

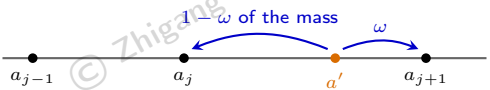
Rounding a' to the nearest node keeps the mass but moves the mean. Young's lottery splits the household between the two neighbouring nodes and keeps both.

If $a_j \le a' < a_{j+1}$, put

$$\omega = \frac{a' - a_j}{a_{j+1} - a_j},$$

and send the fraction $1 - \omega$ of the household's mass to a_j and ω to a_{j+1}.

Why the mean is exact: $(1-\omega)\,a_j + \omega\,a_{j+1} = a'$. Summed over households, tomorrow's capital $K' = \sum \lambda\, a'$ is exact *given the policy*.



Idea

"Where exactly" becomes "which two nodes, in what proportion": a choice between nodes turns into a probability split between them.

Pitfall: the mean, not the spread

Each split adds a little dispersion: the lottery keeps $\mathbb{E}a'$, not the variance. A coarse grid therefore widens the distribution, which is why 6.B1 pushes its histogram on nodes 0.1 apart.

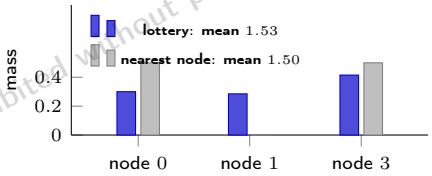
© Zhigang Feng · Redistribution prohibited without permission

Young, "Solving the incomplete markets model with aggregate uncertainty using the Krusell–Smith algorithm and non-stochastic simulations", *JEDC* 34 (2010).

Three Nodes, Three Households

Lab 6's first exercise, by hand: nodes at 0, 1, 3; three households with masses 0.5, 0.3, 0.2 want $a' = 0.4$, 2.5, 2.9.

mass	a'	cell	ω	to lower	to upper
0.5	0.4	$[0, 1]$	0.40	0.300	0.200
0.3	2.5	$[1, 3]$	0.75	0.075	0.225
0.2	2.9	$[1, 3]$	0.95	0.010	0.190



	node 0	node 1	node 3	**mean**
lottery	0.300	0.285	0.415	**1.53**
nearest node	0.5	0	0.5	1.50

Example: check the mean

The true mean is $0.5{\cdot}0.4{+}0.3{\cdot}2.5{+}0.2{\cdot}2.9 = 1.53$. The lottery hits it exactly: $1 \cdot 0.285 + 3{\cdot}0.415 = 1.53$. Rounding sends 2.5 to node 3 and misses by 0.03 — every period, in the same direction if the policy is smooth.

Ledger `core.lottery_example`; **Lab 6**, **cell A4**.

© Zhigang Feng · Redistribution prohibited without permission

The Matrix P_g, Entry by Entry

Number the 1,400 states as one list: a row for today's state, a column for tomorrow's. Each row has at most 14 nonzeros — two asset nodes times seven productivity levels.

The numbering. With $n_\varepsilon = 7$ levels, state (a_i, ε_q) is entry $k = 7i + q$ of the list ($i = 0, \ldots, 199$; $q = 0, \ldots, 6$): the seven levels of node a_0 first, then those of a_1, and so on.

The weight ω. Find the cell holding the choice, j with $a_j \le a'(a_i, \varepsilon_q) < a_{j+1}$ (`np.searchsorted`), then

$$\omega = \frac{a'(a_i, \varepsilon_q) - a_j}{a_{j+1} - a_j} \in [0, 1).$$

The row. For every tomorrow-level $\varepsilon_{q'}$:

$$P_g\big[(i,q),(j,q')\big] = (1-\omega)\,\Pi_{qq'}, \qquad P_g\big[(i,q),(j{+}1,q')\big] = \omega\,\Pi_{qq'}.$$

It sums to $(1-\omega)\sum_{q'}\Pi_{qq'} + \omega\sum_{q'}\Pi_{qq'} = 1$: the household goes *somewhere*.

Example: one row, with two levels

Let $\Pi = \left(\begin{smallmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{smallmatrix}\right)$ and let the household at (a_i, ε_0) choose a' a quarter of the way from a_5 to a_6: $j = 5$, $\omega = 0.25$. Four nonzeros in its row: 0.675 at $(5,0)$, 0.075 at $(5,1)$, 0.225 at $(6,0)$, 0.025 at $(6,1)$ — 0.75 or 0.25 times 0.9 or 0.1; sum 1. Seven levels: fourteen.

At $r = 2\%$: 19,488 nonzeros among 1,960,000 entries; row sums one to 4×10^{-16}. The code never forms the matrix (next frame).

© Zhigang Feng · Redistribution prohibited without permission

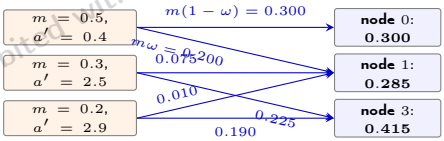
Ledger `core.distribution_2pct; transition_matrix` in `s06_ha.py`.

Way 1: Iterate the Operator

Start everyone at the limit and push the mass forward until it stops moving. This is the power method, and it needs no matrix.

1: $\lambda \leftarrow$ all mass at $a_1 = \underline{a}$, split across ε by π ▷ any start
2: $(j, \omega) \leftarrow$ lottery(a') ▷ once: a' is fixed
3: **repeat**
4: for each q: $m \leftarrow \lambda_q$
5: $t \leftarrow$ `bincount`$(j, m(1-\omega))+$`bincount`$(j{+}1, m\omega)$
6: $\lambda^{\text{new}} \leftarrow \lambda^{\text{new}} + t\,\Pi_q$. ▷ then draw ε'
7: $d \leftarrow \max|\lambda^{\text{new}} - \lambda|$; $\lambda \leftarrow \lambda^{\text{new}}$
8: **until** $d < 10^{-11}$

`bincount`(j, w): entry b is $\sum_{i:\, j_i = b} w_i$ — each weight lands in the bin its index names. Twice per level: lower nodes, then upper. Right: Part 2's three households.



Measured

At $r = 2\%$: **493** sweeps, 0.05 s; at r^*, 596 from the limit, **202** warm-started. Why hundreds: each sweep shrinks the distance by $|\lambda_2| = 0.963$, P_g's second eigenvalue — persistent income, slow mixing.

© Zhigang Feng Redistribution prohibited without permission

Ledger `core.distribution_2pct` (`second_eigenvalue_modulus`), `core.warm_start`; **Lab 6, cell A5**.

Way 2: Solve the Linear System Once

The invariant λ solves 1,400 linear equations, of which one is redundant and must be replaced by the condition that the mass sums to one. Then one solve, no sweeps.

The unknown: $\lambda \in \mathbb{R}^{1400}$, one number per state (a_i, ε_q).

The equations: $\lambda = \lambda P_g$: for each tomorrow-state k, $\lambda_k = \sum_l \lambda_l \, P_g[l,k]$ — the mass arriving equals the mass there. As a system, $(P_g^\top - I)\lambda = 0$.

One is redundant. Every row of P_g sums to one, so the 1,400 equations add up to $0 = 0$: the rank is 1,399 and $\lambda = 0$ solves the system too. Replace one equation by $\sum_k \lambda_k = 1$: $A = P_g^\top - I$ with its last row set to $(1, \ldots, 1)$, $b = (0, \ldots, 0, 1)^\top$, $\lambda = A^{-1} b$.

A dense solve costs order n^3 (4.A2): cheap at 1,400 states, prohibitive at 10^5 — then Way 1, which never forms the matrix.

Measured, $r = 2\%$

Way 2: 0.06 s. Against Way 1: largest gap 2.5×10^{-10}, the iteration's tolerance. The same λ, with no sweeps.

Idea

Way 1 lets the operator move mass until it stops; Way 2 asks linear algebra for the vector it leaves fixed. The redundancy is why a solution exists: a stochastic matrix always has the eigenvalue 1.

© Zhigang Feng — Redistribution prohibited without permission

Ledger `core.distribution_2pct; stationary_distribution(method="solve")` **in** `s06_ha.py: A[-1, :] = 1.0; b[-1] = 1.0; np.linalg.solve(A, b).`

Existence, Uniqueness, Convergence — in Words

2.A3 proved them from three properties of the operator. Here is why each one holds in this economy.

Property (2.A3)	Guarantees	Why it holds here
bounded assets	a compact state	$\beta(1+r) < 1$: wealth has a ceiling (6.A1's net-saving picture)
Feller	existence	the policy is continuous in a; Π is a finite chain
mixing	uniqueness	a run of low ε takes anyone to the limit, a run of high ε takes anyone up: every state communicates with the limit
monotonicity	convergence from any start	a' rises with a, and a higher ε today means a higher ε tomorrow in distribution

© Zhiguo Feng · Redistribution prohibited without permission

Pitfall: 2.A3's two innocent matrices

With $Q_a = I$ every distribution is invariant; with Q_b that swaps two states, iteration from $(1, 0)$ never converges. A solver reports an answer in both cases. Part 3's agreement of three routes, and Part 6's checks, are how you notice.

Monte Carlo: Simulate a Panel — How It Works, and What It Costs

The obvious alternative to a histogram: follow N households. No grid for the distribution and no lottery — and an error that falls only as $1/\sqrt{N}$.

1: N households at $a = \underline{a}$, ε drawn from π ▷ any start
2: **for** $t = 1, \ldots, T$ **do**
3: each household: $a \leftarrow a'(a, \varepsilon)$ ▷ no lottery
4: draw ε' from row ε of Π ▷ one uniform
5: if $t >$ burn-in: record $K_t \leftarrow \frac{1}{N}\sum a$ ▷ a sample from λ
6: **end for**

After the burn-in the N households are a draw from λ: K_t is its mean, any moment a sample average. Stored: N pairs (a, ε) — no grid, no matrix, no lottery.

Why it matters later. The histogram needs a grid over the state; with many or continuous states none fits and the panel is the only route: the original Krusell–Smith, and Sessions 8–9.

The cost, measured at $r = 2\%$

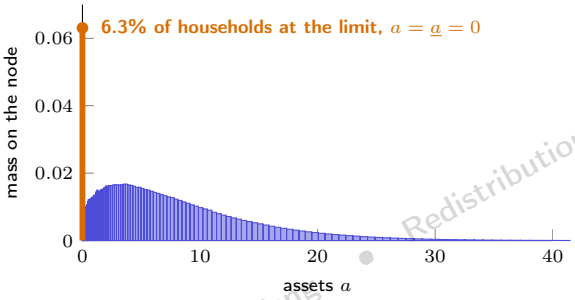
$N \cdot T$ policy evaluations; an error of order $1/\sqrt{N}$. At $T = 1{,}000$ (500 dropped): $N = 10^4$ takes 1.3 s and K_t wobbles by 0.046; $N = 10^5$ takes 12.9 s and wobbles by 0.013 — ten times the households, one more digit. The histogram: $K = 4.656$ to ten digits in 0.05 s.

© Zhuoqing Cheng Redistribution prohibited without permission

Ledger `monte_carlo` (`simulate_panel` in `s06_ha.py`, **How It Works (3)**). **A panel also buries derivatives in noise: Lab 6, B2, ledger** `lottery_grad`. **Keep the histogram wherever it fits.**

The Distribution, Pictured, and Two Checks

The invariant distribution at $r = 2\%$, mass by asset node. Two checks before anyone uses it.



© Zhigang Feng Redistribution prohibited without permission

Reading it

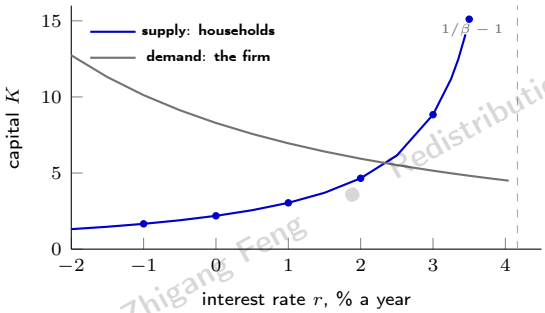
6.3% of households sit at the limit (the orange spike at $a = 0$); mean assets are 4.656 — the capital households supply at $r = 2\%$. Bars widen to the right because the cells do.

Two checks

(1) $\sum \lambda = 1$: here exact to machine precision. (2) Mass on the top node: 0 here. Were it 10^{-3}, the grid, not the economics, would set K: widen it.

Ledger `core.distribution_2pct`, `fig_wealth_nodes_2pct`; **the first 120 of 200 nodes. Lab 6, cell A5.**

Each point of the supply curve is one household solve (6.A1) plus one distribution (Parts 1–4). This is 2.A2's figure, now computed.



Reading it

Supply rises from 1.66 at $r = -1\%$ to 38.6 at 3.95% and has no bound as $r \to 1/\beta - 1$. Demand falls. They cross once.

Each point costs about 0.1 s from scratch. The next frames find the crossing with about twenty of them.

Ledger `core.capital_market, fig_capital_market`; **dots**: **Lab 6's seven points (cell A6)**, **warm-started**. `core.inner_solve_seconds`.

© Zhigang Feng
Redistribution prohibited without permission

5 · Bisection Needs a Bracket With a Sign Change — Check It First

Bisection halves whatever bracket it is given. The check: evaluate d at both ends and refuse to start unless they differ in sign. Without it, a root outside the bracket goes unnoticed.

What the check is. $d(r) = K^s(r) - K^d(r)$, excess supply. Compute d at both ends before any halving. If $d(r_{\text{lo}}) < 0 < d(r_{\text{hi}})$, continuity puts a root inside and every halving keeps one inside; if not, stop and widen the bracket.

Without it. A careless bisection (Lab 6, A7) on $[-2\%, 2\%]$ "converges" to $r = 2.000\%$: the edge of its bracket. Households supply 4.66, the firm demands 5.94 — **the market is 21.6% off**, and nothing in the output says so until the clearing residual is printed.

Here. $d = -64.9$ at $r = -6\%$ and $+105.5$ at 4.1657%: a root is inside, and 20 halvings find it.

Which bracket: the economics

Lower end: $r > -\delta$, or capital is free; the code uses $-\delta + 2\%$.

Upper end: the after-tax return must stay below $1/\beta - 1$; above it wealth has no ceiling and supply is infinite. The code uses $(1/\beta - 1)/(1 - \tau_k) - 10^{-5}$.

A solver that cannot fail still needs a bracket that contains the root. The check costs two inner solves and makes the bracket a tested fact.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `core.careless_bisection`, `core.equilibrium.sign_change`; **4.A1**, "**Bisection: The One That Cannot Fail**".

5 · Another Way to Move r: Follow the Excess Demand

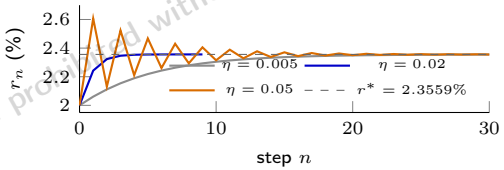
No bracket and no halving: move the price in the direction the market pushes it, by an amount that cannot explode. 2.A2's capital market, as an algorithm.

$$r_{n+1} = r_n + \eta \underbrace{\frac{K^d(r_n) - K^s(r_n)}{K^d(r_n) + K^s(r_n)}}_{g(r_n)\in(-1,1)}.$$

The sign. Demand above supply: the firm wants more capital than households offer, so r rises; the reverse lowers it; at r^* the step is zero.

The normalization. Supply explodes as $r \to 1/\beta - 1$, so a step proportional to $K^d - K^s$ alone could throw r out of the economy. Divided by $K^d + K^s$ the ratio stays in $(-1, 1)$: a step is at most η.

The dial η. Near r^* each step multiplies the distance by $|1 + \eta\, g'(r^*)|$, $g'(r^*) = -36.3$ here: $\eta = 0.02$ keeps 0.27 per step, 0.005 keeps 0.82; above $2/36.3 = 0.055$ the steps grow.



Measured, from $r_0 = 2\%$

$\eta = 0.01$: 24 steps; 0.02: **10** steps, 0.8 s; 0.05: 50, oscillating; 0.10: never settles. Bisection: 20 steps, 3.4 s; every converging run agrees with it to 10^{-5} points.

Ledger `r_update` (`excess_demand_update`, `s06_ha.py`; **steps clipped to the economy's bounds**). **6.B2 damps a path of K the same way.**

© Zhaoyong Hu · Redistribution prohibited without permission

Why Bisection, Not Newton — and Warm Starts

Every evaluation of $d(r)$ costs a full inner solve, so the outer method must need few of them and must never fail.

Why not Newton (4.A1, 4.A2)

$d(r)$ is only piecewise smooth: as r moves, the set of nodes where the limit binds changes discretely, and the lottery's node indices jump. A finite-difference derivative across such a switch is not the derivative of anything, and a Newton step can leave the bracket. Monotonicity of d is not a theorem either.

Bisection needs only a sign: 20 halvings take the bracket from about 10% to 10^{-7}.

Warm starts

Each new r starts from the previous policy and distribution. At r^*: **142** EGM sweeps instead of 356, **202** distribution sweeps instead of 596.

The whole equilibrium: 20 bisections, 3.4 s; with Way 2's dense solve for λ at every price, 2.2 s; along the excess demand with $\eta = 0.02$, 10 steps and 0.8 s.

Ledger `core.equilibrium`, `core.warm_start`, `core.dense_solve_equilibrium`.

© Zhigang Feng
Redistribution prohibited without permission

$r^* = 2.356\%$, 1.81 points below $1/\beta - 1$: precautionary saving, as a number.

interest rate r^*	2.3559%
patience, $1/\beta - 1$	4.1667%
capital K^*	5.639
wage w	1.186
output Y	1.770
K/Y	3.19
saving rate $\delta K/Y$	25.5%

Idea

Without income risk, $r = 1/\beta - 1$ and $K = 4.43$ (the last row two frames on). With uninsurable risk households save more for bad times: capital is 27% higher and the return 1.81 points lower. That is Aiyagari's (1994) result, measured.

Example: how precise

20 bisections; the final bracket is narrower than 10^{-7}, and the market clears to 4.3×10^{-6} of demand.

© Zhigang Feng
Redistribution prohibited without permission

Ledger `core.equilibrium`, `core.precautionary`. **Aiyagari**, "**Uninsured idiosyncratic risk and aggregate saving**", ***QJE*** **(1994)**.

Five numbers before believing an equilibrium. Only the last is expensive, and it is the one that most often changes an answer.

1. **The market clears:** $|K^s - K^d|/K^d = 4.3 \times 10^{-6}$.
2. **The Euler equation holds between nodes:** mass-weighted mean $10^{-4.95}$; worst $10^{-1.79}$, in the cell with the kink.
3. **The grid is wide enough:** mass on the top node is zero.
4. **Mass is conserved:** $|\sum \lambda - 1| = 1.1 \times 10^{-16}$.
5. **The answer survives refinement:** the table.

n_a	r^*, %	K^*	Gini	Euler
100	2.3436	5.649	0.537	$10^{-4.38}$
200	2.3559	5.639	0.534	$10^{-4.95}$
400	2.3594	5.637	0.534	$10^{-5.61}$
800	2.3602	5.636	0.534	$10^{-6.22}$

From 200 to 800 nodes r^* moves by 0.004 points and the Gini not at all: 200 nodes are enough for these numbers.

Ledger `core.five_checks`, `core.refinement` **(Euler: mass-weighted mean residual).**

© Zhigang Feng · Redistribution prohibited without permission

6 · What It Gets Right, What It Misses — and It Is Not the Calibration

Uninsurable earnings risk and a borrowing limit alone make wealth twice as unequal as earnings. The top is missing, and no parameter here brings it back.

	model	U.S.
Gini, earnings	0.248	
Gini, wealth	0.534	≈ 0.82
top 10% share	34.0%	
top 1% share	5.5%	$\approx \frac{1}{3}$
at the limit	2.9%	

U.S. figures as cited in 1.A2 (SCF).

economy	r^*, %	Gini	at limit	top 1%
persistence 0.60	3.08	0.41	0.5%	3.9%
persistence 0.90	2.36	0.53	2.9%	5.5%
persistence 0.95	2.44	0.60	10.2%	6.7%
risk s.d. 0.30	3.31	0.52	5.2%	5.1%
risk s.d. 0.60	1.39	0.55	2.5%	5.9%
limit -1	2.54	0.61	2.3%	6.0%
limit -2	2.68	0.68	1.8%	6.4%
no risk	4.17	—	—	—

Persistence rows hold the unconditional s.d. of $\log \varepsilon$ fixed. 2.A1 promised a number for this; here it is: persistence moves the constrained share from 0.5% to 10.2%.

Ledger `core.five_checks`, `core.statics`; **"at limit" is the mass at the borrowing limit, which moves with the limit itself.**

© Zhigang Feng
Redistribution prohibited without permission

The Whole Algorithm on One Frame

The three nested problems of 6.A1, with every line now built and measured.

1: grid, ε, $\Pi \leftarrow$ power grid, Rouwenhorst ▷ 6.A1, Part 1
2: $r_{\text{lo}} \leftarrow -\delta + 0.02$; $r_{\text{hi}} \leftarrow (1/\beta - 1)/(1 - \tau_k) - 10^{-5}$
3: evaluate $d(r_{\text{lo}})$, $d(r_{\text{hi}})$; stop unless $d(r_{\text{lo}}) < 0 < d(r_{\text{hi}})$ ▷ Part 5
4: **while** $r_{\text{hi}} - r_{\text{lo}} > 10^{-7}$ **do**
5: $r \leftarrow (r_{\text{lo}} + r_{\text{hi}})/2$; $(K^d, w) \leftarrow$ the firm at r
6: $c, a' \leftarrow$ the household by EGM, warm-started ▷ 6.A1
7: $\lambda \leftarrow$ invariant distribution by iteration, warm-started ▷ Parts 1–3
8: $d \leftarrow \sum \lambda\, a - K^d$; if $d > 0$: $r_{\text{hi}} \leftarrow r$, else $r_{\text{lo}} \leftarrow r$
9: **end while**
10: report r^*, K^*, w, and the five checks ▷ Part 6

▷ Or replace Lines 2–4 and 8 by $r \leftarrow r + \eta\,(K^d - K^s)/(K^d + K^s)$, until $|K^d - K^s|/K^d < \epsilon$ (Part 5).

`solve_equilibrium` in `s06_ha.py`; **Lab 6, cell A8**.

Ten lines, three fixed points. Lines 6 and 7 are the expensive ones; the warm starts are why they are cheap after the first price.

© Zixiang Feng · Redistribution prohibited without permission

How It Works (1): The Lottery, One Push, and the Matrix

```python
def lottery(ap, grid):
    """Young (2010): split the mass at a' between the two neighbouring nodes."""
    a1 = np.clip(ap, grid[0], grid[-1])                          # a' inside [a_1, a_N]
    j = np.clip(np.searchsorted(grid, a1, side="right") - 1, 0, grid.size - 2)  # a_j <= a' < a_j+1
    return j, (a1 - grid[j]) / (grid[j + 1] - grid[j])           # j, and omega: the share sent up
```

```python
def push_forward(lam, j, w_hi, Pi):
    """One application of the operator on measures: lambda_t -> lambda_{t+1}."""
    na, ne = lam.shape
    out = np.zeros_like(lam)
    for q in range(ne):                                          # today's productivity level
        m = lam[:, q]                                            # mass at each node, given e_q
        tmp = (np.bincount(j[:, q], weights=m * (1.0 - w_hi[:, q]), minlength=na)   # 1-omega to a_j
               + np.bincount(j[:, q] + 1, weights=m * w_hi[:, q], minlength=na))    # omega to a_j+1
        out += np.outer(tmp, Pi[q, :])                           # then e_q -> e_q' with Pi[q, q']
    return out
```

```python
def transition_matrix(j, w_hi, Pi):
    """The same operator as a dense matrix over (a, e) pairs, row-stochastic."""
    na, ne = j.shape
    P = np.zeros((na * ne, na * ne))                             # one row, one column per (a_i, e_q)
    rows = np.arange(na)
    for q in range(ne):                                          # today's e_q ...
        for qn in range(ne):                                     # ... and tomorrow's e_q'
            lo, hi = j[:, q] * ne + qn, (j[:, q] + 1) * ne + qn    # the two columns k'
            np.add.at(P, (rows * ne + q, lo), (1.0 - w_hi[:, q]) * Pi[q, qn])   # row k = ne i + q
            np.add.at(P, (rows * ne + q, hi), w_hi[:, q] * Pi[q, qn])           # omega Pi[q, q']
    return P
```

© Zhiyang Feng · Redistribution prohibited without permission

How It Works (2): The Invariant Distribution, Two Ways

```python
def stationary_distribution(ap, grid, Pi, pi, method="solve", lam_init=None,
                            tol=1e-11, max_iter=200000):
    """Invariant distribution over (a, e): by a linear solve, or by iteration."""
    na, ne = ap.shape
    j, w_hi = lottery(ap, grid)                              # where the policy sends each node
    if method == "solve":                                    # Way 2: (P' - I) lambda = 0
        A = transition_matrix(j, w_hi, Pi).T - np.eye(na * ne)   # n_a n_e equations, rank one less
        A[-1, :] = 1.0                                       # replace one equation by sum = 1
        b = np.zeros(na * ne); b[-1] = 1.0                   # right-hand side: 0, ..., 0, 1
        lam = np.maximum(np.linalg.solve(A, b), 0.0).reshape(na, ne)   # one dense solve
        return lam / lam.sum(), 1, 0.0
    lam = np.zeros((na, ne)); lam[0, :] = pi                 # everyone starts at the limit
    if lam_init is not None:                                 # ... or at a warm start
        lam = lam_init.copy()
    diff, it = np.inf, 0
    for it in range(1, max_iter + 1):                        # Way 1: lambda <- lambda P
        new = push_forward(lam, j, w_hi, Pi)
        diff = float(np.abs(new - lam).max())
        lam = new
        if diff < tol:                                       # it no longer moves: invariant
            break
    return lam / lam.sum(), it, diff
```

© Zhirang Feng • Redistribution prohibited without permission

`method="solve"` is Way 2, the default branch is Way 1. Lab 6, cell A5 times both.

How It Works (3): Simulate a Panel, and Follow the Excess Demand

```python
def simulate_panel(ap, grid, Pi, pi, n=10000, T=1000, burn=500, seed=0):
    """Monte Carlo: N households, each a draw of e' from Pi and a' = a'(a, e) read exactly."""
    rng = np.random.default_rng(seed)
    cum = np.cumsum(Pi, axis=1); cum[:, -1] = 1.0              # rows of Pi as cumulative sums
    a = np.zeros(n)                                            # everyone starts at the limit ...
    q = rng.choice(pi.size, size=n, p=pi)                      # ... with e drawn from pi
    K = np.empty(T - burn)
    for t in range(T):
        for s in range(pi.size):                               # by productivity group
            here = q == s
            a[here] = np.interp(a[here], grid, ap[:, s])       # a' between the nodes: no lottery
        q = (rng.random(n)[:, None] > cum[q]).sum(axis=1)      # one uniform draws e' from Pi[q, :]
        if t >= burn:
            K[t - burn] = a.mean()                             # K_t = mean assets of the sample
    return K, a, q
```

```python
def excess_demand_update(p: Params, r0=0.02, eta=0.02, tol=1e-5, max_iter=100,
                         dist="iterate"):
    """Move r by the excess demand for capital over K_d + K_s: a step is at most eta."""
    grid, e, Pi, pi = economy(p)
    r_lo, r_hi = -p.delta + 0.01, r_upper(p) - 1e-5            # outside: no bounded solution
    r, sol, path = r0, None, []
    for n in range(1, max_iter + 1):
        sol = capital_supply(r, p, grid, e, Pi, pi, dist=dist,  # households at r ...
                             c_init=None if sol is None else sol["c"],
                             lam_init=None if sol is None else sol["lam"])   # ... warm-started
        Kd, Ks = sol["K_demand"], sol["K_supply"]
        gap = (Kd - Ks) / (Kd + Ks)                            # excess demand, in (-1, 1)
        path.append((r, gap))
        if abs(Kd - Ks) / Kd < tol:                            # the market clears: stop
            break
        r = float(np.clip(r + eta * gap, r_lo, r_hi))          # demand > supply: r rises
    return dict(r=r, iterations=n, path=path, K_demand=Kd, K_supply=Ks, sol=sol)
```

Redistribution prohibited without permission

How It Works (4): One Point of the Supply Curve, and the Bisection

```
def capital_supply(r, p: Params, grid, e, Pi, pi, c_init=None, dist="iterate",
                   lam_init=None):
    """Households' capital at a pre-tax return r; the tax is rebated lump-sum."""
    kl, w = firm(r, p)                                          # K/L the firm wants at r, and w
    r_net, transfer = (1.0 - p.tau_k) * r, p.tau_k * r * kl     # after-tax return; the rebate
    ap, c, it, diff = solve_household(r_net, w, transfer, grid, e, Pi, p,
                                      c_init=c_init)            # 6.A1: the policy at these prices
    lam, it_d, _ = stationary_distribution(ap, grid, Pi, pi, method=dist,
                                           lam_init=lam_init)   # Part 3: the population
    Ks = float((lam.sum(axis=1) * grid).sum())                  # K supplied = mean assets
    return dict(r=r, r_net=r_net, w=w, transfer=transfer, K_demand=kl, K_supply=Ks,
                ap=ap, c=c, lam=lam, egm_sweeps=it, dist_sweeps=it_d)
```

```
lo = capital_supply(r_lo, p, grid, e, Pi, pi, dist=dist)   # both ends of the bracket
hi = capital_supply(r_hi, p, grid, e, Pi, pi, dist=dist)
d_lo, d_hi = lo["K_supply"] - lo["K_demand"], hi["K_supply"] - hi["K_demand"]  # excess supply
if not (d_lo < 0.0 < d_hi):                                # no sign change: refuse to run
    raise RuntimeError(f"no sign change on [{r_lo:.5f}, {r_hi:.5f}]: "
                       f"excess supply {d_lo:.4f}, {d_hi:.4f}")
sol, n = hi, 0
while r_hi - r_lo > tol:                                   # halve until the bracket is tiny
    n += 1
    sol = capital_supply(0.5 * (r_lo + r_hi), p, grid, e, Pi, pi,   # the midpoint ...
                         c_init=sol["c"], dist=dist, lam_init=sol["lam"])  # ... warm-started
    d = sol["K_supply"] - sol["K_demand"]                  # excess supply there
    if verbose:
        print(f"   {n:2d}  r = {sol['r']:.6f}   excess supply = {d:+.6f}")
    if d > 0:                                              # supply > demand: r is too high
        r_hi = sol["r"]
    else:                                                  # demand > supply: r is too low
        r_lo = sol["r"]
```

© Zhigang Feng
Redistribution prohibited without permission

Takeaways

- **Part 1.** Given the policy, the population is the fixed point of a linear map: $\lambda = \lambda P_g$ on 1,400 states, each a pair (a_i, ε_q).
- **Part 2.** Young's lottery splits a household between the two nodes around a', with weight $\omega = (a' - a_j)/(a_{j+1} - a_j)$ on the upper one, and keeps the mean exact; rounding does not.
- **Part 3.** Iterate (493 sweeps, the speed set by $|\lambda_2| = 0.963$; `bincount` twice per level), or solve the linear system once with one equation replaced by $\sum \lambda = 1$: the same λ to 10^{-10}.
- **Part 4.** One answer exists and iteration finds it from anywhere (2.A3). A simulated panel does the same job on any state space, at the price $1/\sqrt{N}$: ten times the households, one more digit.
- **Part 5.** The price comes from a bisection that first checks the sign change, or from steps along the normalized excess demand: 10 steps at $\eta = 0.02$ against 20 halvings, with η the dial.
- **Part 6.** $r^* = 2.356\%$, 1.81 points below $1/\beta - 1$; five checks pass. Wealth is twice as unequal as earnings, and no parameter here produces the top of the distribution.

© Zhixiang Feng. Redistribution prohibited without permission

Bridge: Now Let the Distribution Move

Block A solved an economy in which nothing aggregate ever happens: the distribution sits still, so prices are numbers.

Block B takes that away twice. First an aggregate shock that keeps coming: prices depend on the whole distribution, which becomes a state households must forecast. Then a single surprise — a tax reform — after which the distribution travels from one steady state to another.

Next: 6.B1 and 6.B2

Krusell–Smith: the household of 6.A1 with two more states, inside a loop on a forecast rule — a nested fixed point. Then transitions after an MIT shock, and the life cycle.

Homework 2, C1 is this deck's loop on the quarterly calibration of Homework 1.

© Zhigang Feng · Redistribution prohibited without permission